

Agentic Merchant OS

Architecture Overview

Razorpay Buildathon — Track 01: AI Growth & Agentic Commerce

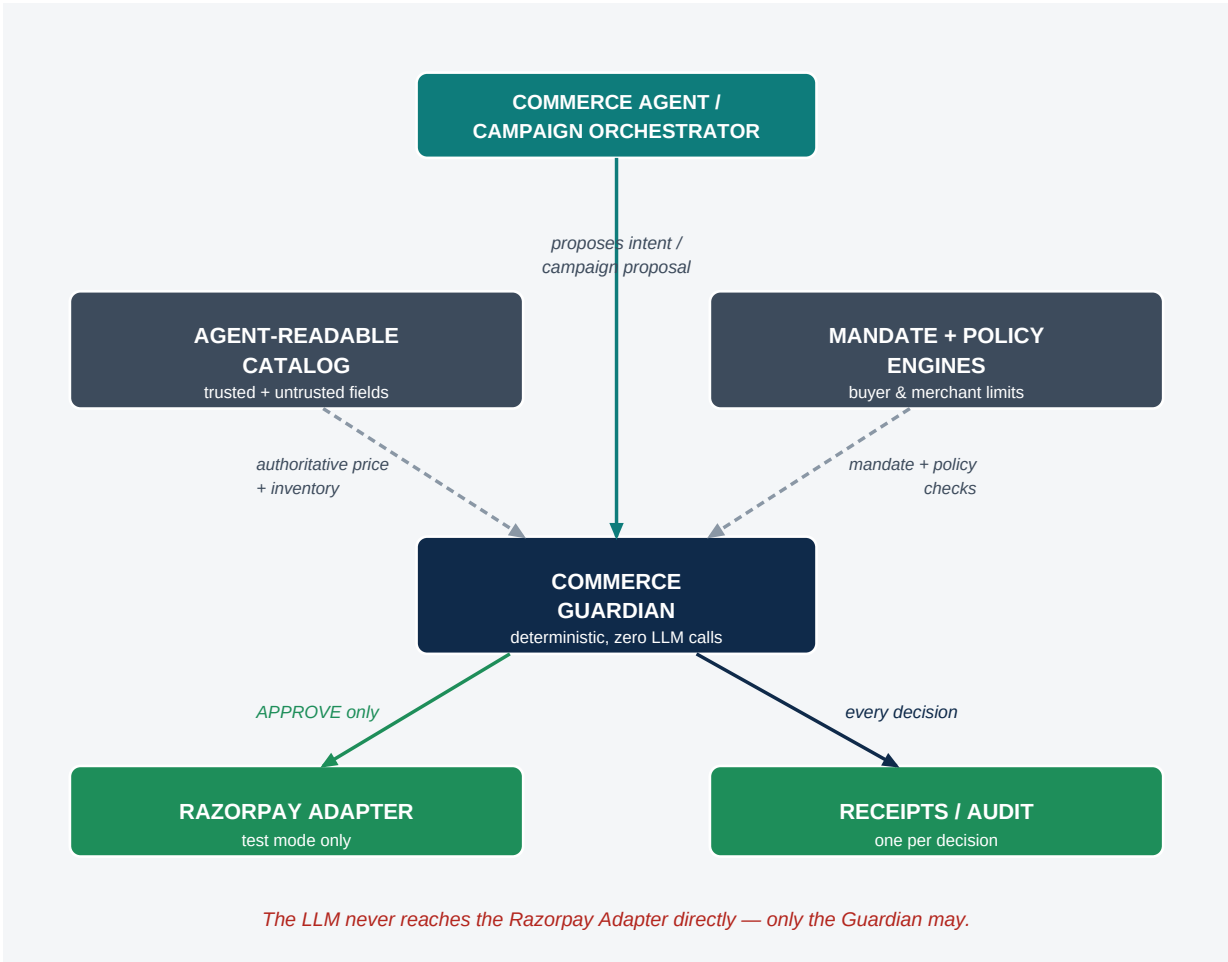


Figure 1. The LLM proposes; the Commerce Guardian decides. Nothing reaches Razorpay without passing the Guardian.

Track objective	Grow merchant revenue and make the merchant transactable by an AI buyer — every money action explainable, bounded, and gated.
Mode	Razorpay test-mode only. Not production software.
Core principle	AI Agent → Transaction Intent → Commerce Guardian (deterministic) → APPROVE / BLOCK / REQUIRE_CONFIRMATION → Razorpay.

1. System Overview

Agentic Merchant OS is a single FastAPI backend and a Next.js frontend, modular internally by domain package, backed by SQLite (Postgres-ready). There is no microservice split, no message bus, no external infrastructure beyond Razorpay's test-mode API and an LLM provider. The system demonstrates both sides of Track 01 in one coherent flow: an AI Commerce Agent makes the merchant transactable by a buyer, and an AI Campaign Orchestrator grows revenue — both gated by the same deterministic Commerce Guardian.

2. Component Map

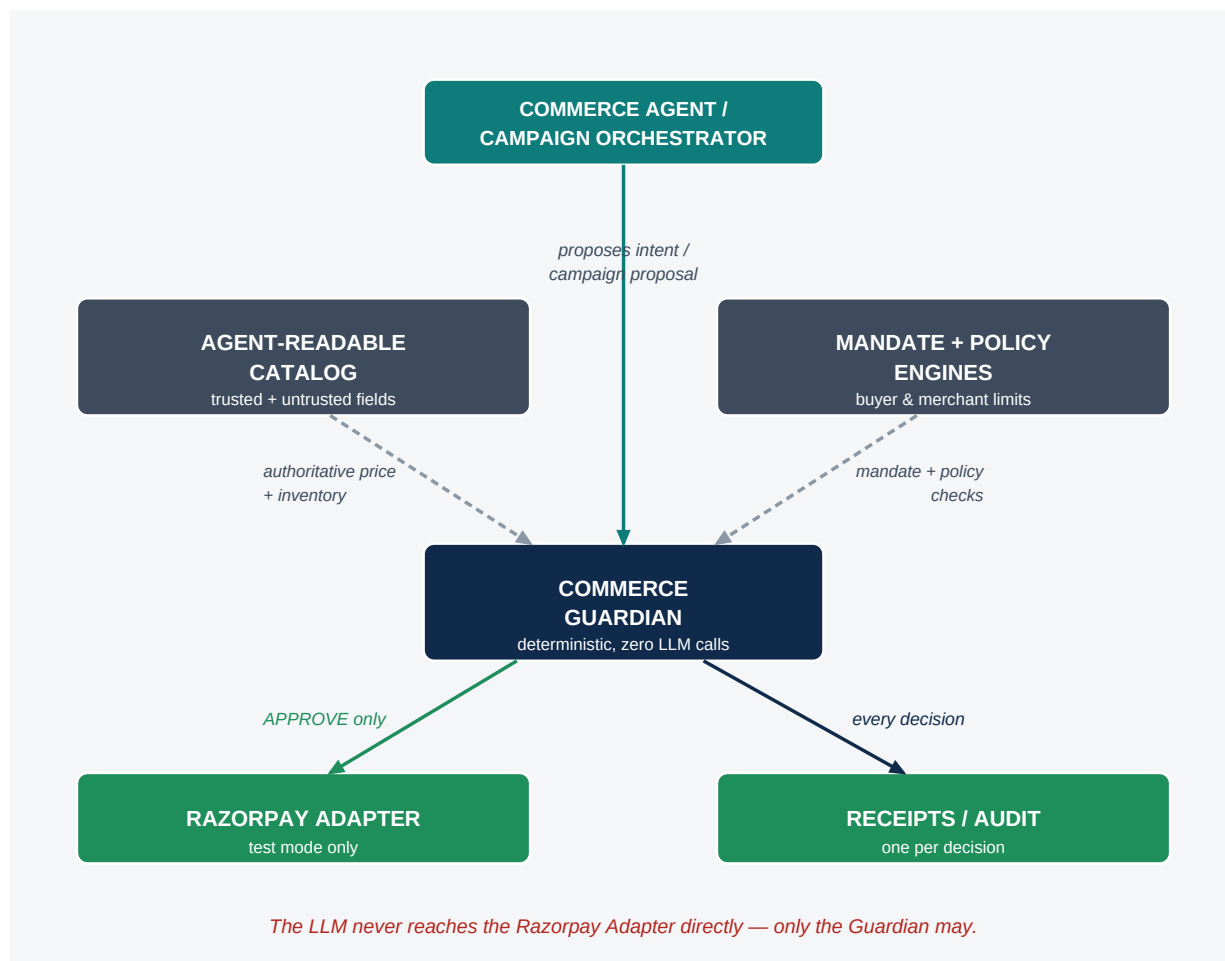


Figure 1 (repeated for reference). Solid arrows = the authorized path to money movement. Dashed arrows = read-only lookups.

3. Component Responsibilities

Component	Responsibility
Commerce Agent	Discovery, cart, upsell, proposes Transaction Intent. Never calls Razorpay.
Campaign Orchestrator	Objective → bounded proposal. Never executes transactions itself.
Agent-Readable Catalog	Trusted structured fields (price, inventory) vs. untrusted free text (description, reviews).
Mandate Engine	Buyer's structured spending authorization (max amount, quantity, categories, expiry).
Merchant Policy Engine	Merchant's structured constraints (max discount, min margin, campaign budget).
Commerce Guardian	Deterministic pipeline. The only component allowed to call Razorpay.
Razorpay Adapter	Thin wrapper: create order, verify payment, handle webhook, refund. Test mode only.
Receipts / Audit	One immutable, replayable Decision Receipt per Guardian decision — approved or not.

4. Data Flow — Buyer Purchase

The buyer chats with the Commerce Agent, which searches the catalog and builds a cart. On checkout, the Agent assembles a Transaction Intent from real cart state — never from LLM free text — and submits it to the Guardian. The Guardian re-reads authoritative catalog state, validates the buyer's Mandate and the merchant's Policy, and only then may call the Razorpay Adapter.

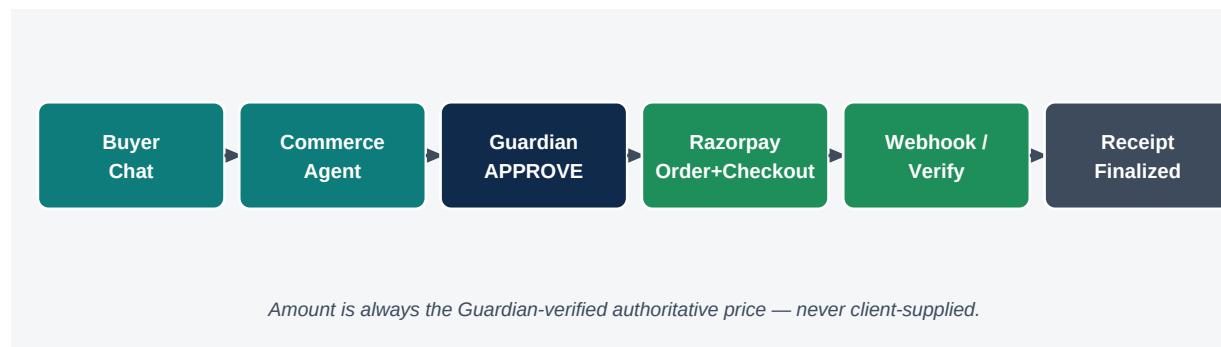


Figure 2. Payment flow. Every step after the Guardian's APPROVE is auditable back to one Receipt.

5. Guardian Decision Flow

The Guardian is a deterministic pipeline — zero LLM calls anywhere in this path. It short-circuits on the first hard failure but always records every check attempted. A price increase or inventory drift since the buyer last saw the product routes to REQUIRE_CONFIRMATION rather than a silent charge at the old price.

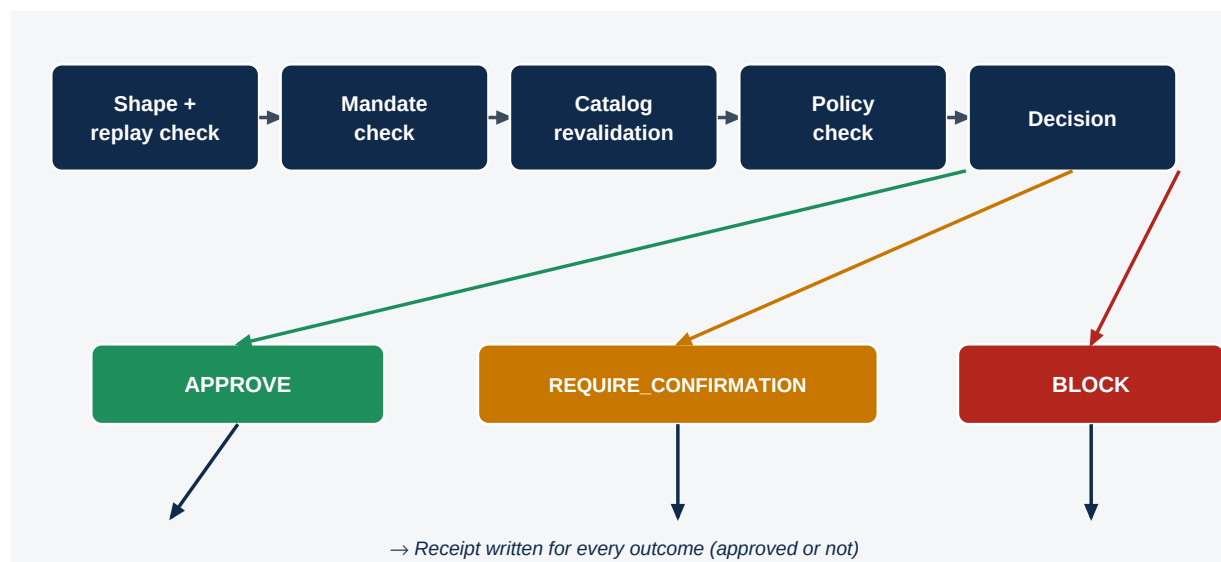


Figure 3. Guardian pipeline. Mandate check, catalog revalidation, and policy check all run against freshly-read, structured data only.

6. Agent Flow — What the LLM Actually Controls

The Commerce Agent's LLM calls a small, fixed set of tools: `search_catalog`, `get_product`, `add_to_cart`, `remove_from_cart`, `get_cart`, `request_checkout`. The Transaction Intent sent to the Guardian is assembled by deterministic code from real CartItem rows — the LLM can influence *which* tool is called and *how it's phrased to the buyer*, but it cannot hand-write the object that authorizes spend. This single architectural choice is what makes catalog prompt injection structurally unable to reach a payment: even fully adversarial catalog text cannot change a number the LLM never gets to write.

7. Catalog Trust Boundary

Field	Trust level	Guardian-usable?
SKU, price, inventory, currency, category, cost	Trusted	Guardian reads directly
variants, bundle_relationships, shipping_info, return_policy	Trusted (structured)	Display + upsell logic
description, marketing_copy, reviews	Untrusted (free text)	Display only — never a Guardian input

8. Decision Semantics

Decision	Trigger	Effect
APPROVE	All mandate + policy + revalidation checks pass	Razorpay order created
BLOCK	A hard constraint failed (limit, stock, expiry, duplicate)	No Razorpay call; Receipt still written
REQUIRE_CONFIRMATION	A soft discrepancy (e.g. price rose since discovery)	Buyer/merchant must explicitly confirm; pipeline re-runs in full

9. Campaign Flow — Revenue Growth Side

A merchant states an objective in plain language. The Campaign Orchestrator's LLM proposes a bounded campaign — eligible SKUs, discount, budget, duration — grounded in real catalog and historical order data. The proposal is validated by the same Guardian/Policy engine used for buyer checkout, not a separate system. Once approved, activation only writes static offer data; the LLM never executes a transaction. Every subsequent purchase under the campaign is still individually re-gated by the Guardian, which also enforces the remaining campaign budget in real time and auto-pauses the campaign — never the underlying purchase — if it would be exceeded.

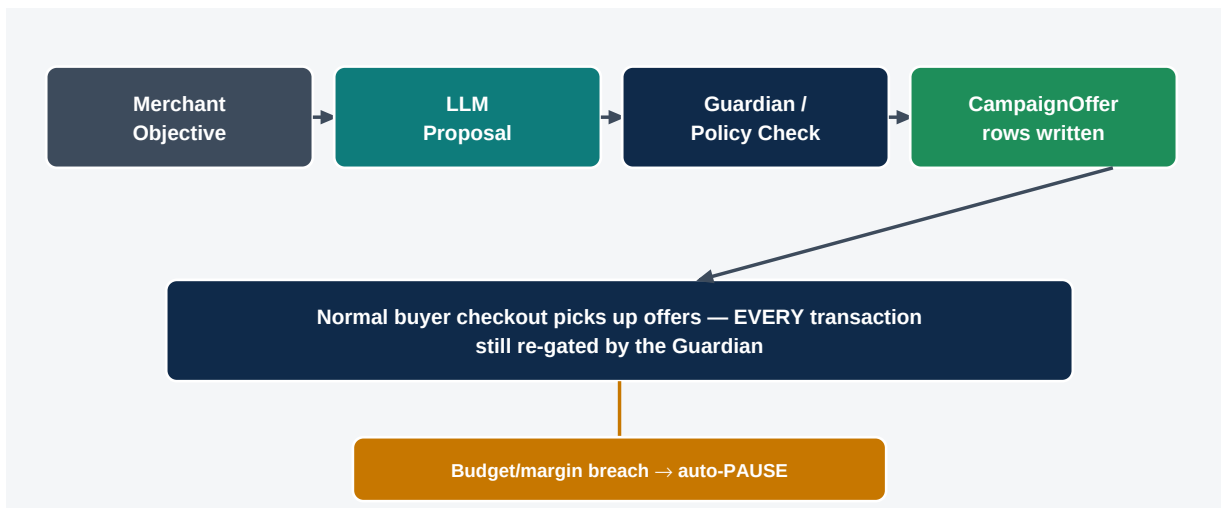


Figure 4. Campaign lifecycle. This is not an autonomous loop — the LLM's involvement is confined to one proposal-generation step.

10. Why This Isn't Two Systems

Both the buyer-purchase path and the campaign path terminate in the same Guardian pipeline, the same Mandate/Policy checks, and the same Receipt schema. A campaign offer is, to the checkout flow, indistinguishable from any other merchant-defined offer — it is validated the same way, every time it's used.

11. Security Boundaries

The primary defense against catalog prompt injection is structural, not behavioral: the Guardian never reads free-text fields, and the Transaction Intent is built by code, not by LLM free text. A secondary heuristic scanner flags suspicious catalog content for visibility — it never gates a decision by itself. The threat model below covers 14 attack classes; each has a deterministic defense and a test case.

Attack class	Defense
Injection in description/reviews	Guardian ignores free text; discount/qty always re-validated against structured data
Agent exceeds buyer mandate	check_mandate() re-run server-side, independent of Agent behavior
Price/inventory drift before payment	Authoritative re-fetch at evaluation time; charged amount is never the observed price
Unauthorized discount	Only active, policy-validated Offer rows are honored
Campaign over budget / under margin	Per-SKU margin check at proposal time; per-transaction budget check at checkout time
Duplicate payment / stale intent replay	intent_id uniqueness + 2-minute expiry window
Webhook forgery	Signature verified before any state change; idempotent on event id
LLM hallucinated price	Intent built from real CatalogSnapshot rows, never LLM free text

Full 14-item threat model with attack/risk/defense/location/test detail is in 13_THREAT_MODEL.md.

12. Failure Handling — Demo-Verified

The buildathon demo includes two required, reproducible failure scenarios:

Failure 1 — Malicious Catalog Content

A seeded product's description contains an instruction attempting to make the AI ignore the buyer's spending limit. The Guardian blocks the transaction on the real mandate check — not a keyword match — and the heuristic scanner's independent flag is also visible on the resulting Receipt, showing the system noticed the attack even though that isn't what stopped it.

Failure 2 — Price Change Mid-Flow

A product's price is changed by the merchant after it was added to a buyer's cart. At checkout, the Guardian's fresh authoritative read detects the mismatch and returns `REQUIRE_CONFIRMATION`, showing both the observed and current price. The buyer must explicitly confirm before any charge is attempted, and the discrepancy is permanently recorded on the Receipt.

Both failures produce a complete Decision Receipt — the audit trail covers blocked and confirmed-pending attempts, not only successful payments.

13. Additional Failure Modes Covered

- Duplicate checkout submission → second attempt blocked as a replay
- Expired buyer mandate → BLOCK: `no_active_mandate`
- Campaign discount below merchant's minimum margin → SKU excluded from campaign
- Inventory hits zero mid-cart → BLOCK: `insufficient_inventory`
- Forged webhook signature → rejected before any processing

14. Technology Stack

Layer	Choice
Frontend	Next.js 14 (App Router), TypeScript, Tailwind
Backend	FastAPI (Python 3.11+), single service, modular by domain package
Database	SQLite + SQLAlchemy 2.0 (async); connection-string swap to Postgres later
AI Provider	Provider abstraction; Gemini default, Groq/OpenRouter swap-ready
Payments	Razorpay Python SDK, test mode only
Auth	Bearer/session token (JWT), minimal by design
Testing	pytest (backend, 20+ Guardian test-matrix cases), scripted end-to-end scenarios

15. Explicitly Not Used

Kubernetes, Kafka, Redis, event buses, vector databases, and any microservice split. A single, well-modularized service is more debuggable for a solo developer and more parallelizable across coding agents — each package has a clear owner and a mechanically-enforced import boundary (commerce_agent and campaign packages are statically prevented from importing the Razorpay adapter).

16. Deployment Architecture

The MVP is required to run entirely on localhost — no cloud dependency beyond the Razorpay test-mode API and the chosen LLM provider. A future AWS deployment (containerized services, RDS Postgres, a real scheduler for campaign monitoring, secrets management) is documented here for completeness but is explicitly out of scope for this submission.

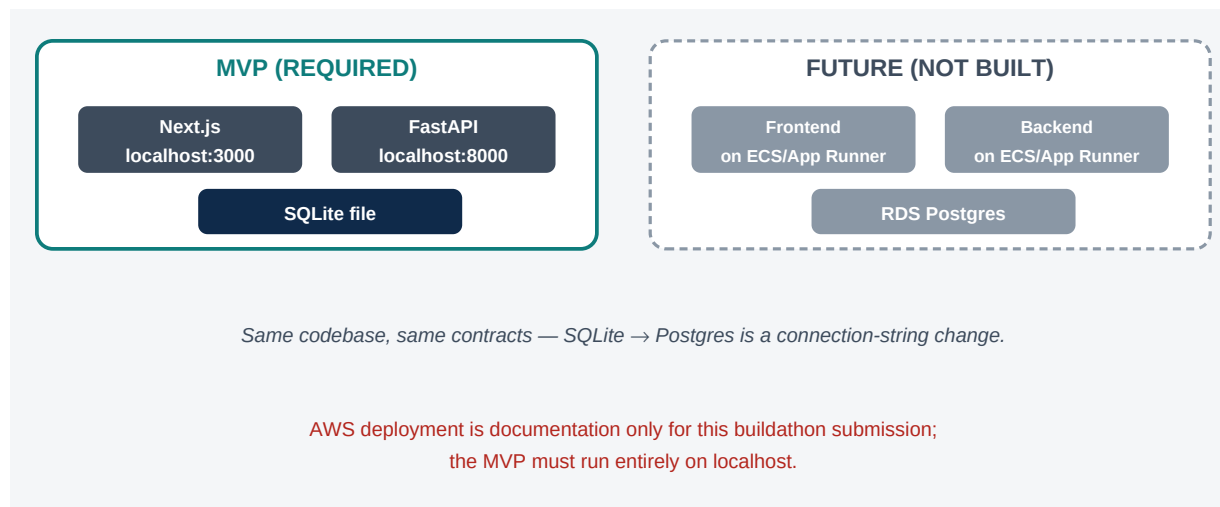


Figure 5. MVP (left, required) vs. a documented future deployment path (right, not built).

17. The Ten Architectural Rules

1	LLM output is never trusted for payment authorization.
2	Catalog content is never authorization.
3	All money actions pass through the Guardian.
4	Current authoritative product state is verified before every payment.
5	Merchant campaign actions must obey merchant-defined policies.
6	No hardcoded fake revenue metrics anywhere in the system.
7	No unnecessary microservices or infrastructure.
8	Every implementation feature has tests.
9	Every money action has an audit/decision receipt.
10	The MVP runs locally, without AWS, as a prerequisite.